

Relatório do Laboratório

1.1 Integrantes

- Osean Pinto Cordeiro do Nascimento (2025015571)
- Erika Andreina Mejias (2025015123)

1.2 Link do repositório

Link:

https://github.com/CoffeeComLeite/AOC_OseanErika_UFRR_LabCircuitos_2026 .

1.3 Introdução com objetivos

Esse relatório faz parte do laboratório da disciplina DCC301 (Arquitetura e Organização da Computadores) da Universidade Federal de Roraima, com o objetivo principal de integrar os estudantes à parte prática da disciplina, segmentada em objetivos de aprendizagem apresentados pelo professor Herbert Rocha.

A finalidade desse relatório se dá por explicar, registrar, processar e sintetizar informações do nosso trabalho em todas as suas etapas.

2. Objetivos de aprendizagem

Nº	Nível (Bloom)	Objetivo
OA1	Compreender	Explicar por que vários dispositivos podem compartilhar um mesmo barramento e qual é o papel do decodificador de endereços nesse arranjo.
OA2	Aplicar	Montar, em Logisim-Evolution, um módulo de memória de 64 bytes com quatro regiões selecionadas por um decodificador 2→4.
OA3	Analisar	Decompor endereços de 6 bits em rótulo, índice e deslocamento e classificar cada acesso de um traço como acerto, falta compulsória ou falta por conflito.
OA4	Aplicar	Calcular a taxa de acertos e o tempo médio de acesso à memória a partir da planilha do traço.
OA5	Aplicar	Implementar uma máquina de estados de cinco estados que gere os sinais de controle de um caminho de dados multiciclo.
OA6	Analisar	Minimizar por mapas de Karnaugh as funções de próximo estado e derivar as equações booleanas correspondentes.
OA7	Criar	Escrever o microprograma de oito bits equivalente à unidade de controle cabeada e compará-lo com ela.

Tabela 2 — Objetivos de aprendizagem.

Figura 1 – Tabela de objetivos de aprendizagem, de autoria do professor

2.1 Módulo de Memória de 64 Bytes (parte1_memoria.circ)

O subsistema de memória possui um espaço de endereçamento de 64 bytes (barramento de 6 bits, A5..A0), estruturado em quatro regiões contíguas e exclusivas de 16 bytes. A decodificação de bloco é realizada a partir dos 2 bits mais significativos (A5 e A4) através de um decodificador 2→4.

2.1.2 Mapa de Memória de 64 Bytes

Faixa (Hex)	Faixa (Binário)	Tamanho	Dispositivo / Função	Sinal de Seleção
0x00 – 0x0F	000000 a 001111	16 bytes	Memória ROM de Programa	CS_ROM
0x10 – 0x1F	010000 a 011111	16 bytes	Memória RAM de Dados	CS_RAM
0x20 – 0x2F	100000 a 101111	16 bytes	Banco de Registradores	CS_REG
0x30 – 0x3F	110000 a 111111	16 bytes	Interface de Entrada/Saída (E/S)	CS_IO

2.1.3 Tabela Verdade dos Sinais de Seleção de Chip (CS)

A decodificação de bloco utiliza os bits A5 e A4. A tabela verdade a seguir demonstra que exatamente um sinal CS fica ativo (=1) em cada combinação, garantindo exclusão mútua no barramento:

A5	A4	Faixa (Hex)	CS_ROM	CS_RAM	CS_REG	CS_IO	Dispositivo Ativo
0	0	0x00 – 0x0F	1	0	0	0	ROM de Programa
0	1	0x10 – 0x1F	0	1	0	0	RAM de Dados
1	0	0x20 – 0x2F	0	0	1	0	Banco de Registradores
1	1	0x30 – 0x3F	0	0	0	1	Interface de E/S

Lógica de Decodificação: Equações Booleanas de Seleção:

- $CS_ROM = \text{NOT}(A5) \text{ AND } \text{NOT}(A4)$
- $CS_RAM = \text{NOT}(A5) \text{ AND } A4$
- $CS_REG = A5 \text{ AND } \text{NOT}(A4)$
- $CS_IO = A5 \text{ AND } A4$

Conforme demonstrado na Tabela Verdade, para qualquer combinação binária do barramento A5A4, exatamente um dos quatro sinais CS assume nível lógico 1. A multiplexação do barramento de dados de saída de 8 bits é realizada por um MUX 4:1 comandado por A5A4.

2.1.4 Detecção de Erro por Paridade Ímpar na RAM

Para garantir a integridade dos dados na RAM (0x10–0x1F), cada palavra de 8 bits é acompanhada por 1 bit de paridade ímpar mantido em uma RAM auxiliar de 16x1 bit. Na escrita, o bit de paridade P_calculado é gerado e armazenado. Na leitura, a paridade do dado lido é recalculada (P_recalculado) e comparada via porta XOR com o bit armazenado (P_armazenado).

A sinalização de erro é mascarada com o sinal CS_RAM para evitar falsos alarmes durante o acesso a outras regiões de memória:

- **Equação de Erro:** $\text{ERRO_PARIDADE} = (P_calculado \text{ XOR } P_armazenado) \text{ AND } CS_RAM$

2.1.5 Memória Cache de Mapeamento Direto (parte1_cache.circ)

2.1.5.1 Demonstração Matemática da Divisão do Endereço

Para um espaço de endereçamento de 6 bits (A5..A0) e uma memória cache com 4 linhas e blocos de 2 bytes (capacidade de 8 bytes de dados):

- **1. Deslocamento (Offset):** deslocamento = $\log_2(\text{tamanho do bloco}) = \log_2(2 \text{ bytes}) = 1 \text{ bit}$ --> Campo A0
- **2. Índice (Index):** índice = $\log_2(\text{numero de linhas}) = \log_2(4 \text{ linhas}) = 2 \text{ bits}$ --> Campo A2 A1
- **3. Rótulo (Tag):** rótulo = Total_Bits - Índice - Deslocamento = $6 - 2 - 1 = 3 \text{ bits}$ --> Campo A5 A4 A3

Fatiamento Prático: Exemplo Demonstrativo (Endereço 0x22 = 100010_2):

- Rótulo (A5 A4 A3): 100_2
- Índice (A2 A1): 01_2 (Linha 1 da cache)
- Deslocamento (A0): 0_2 (Byte 0 do bloco)

2.1.5.2 Lógica de Comparação e Decisão de Acerto (Hit)

A comparação entre o rótulo do endereço requisitado (T2 T1 T0) e o rótulo armazenado na linha indexada (S2 S1 S0) é sintetizada exclusivamente através de portas AND, NOT e OR:

- $\text{XOR}_i = (T_i \text{ AND NOT}(S_i)) \text{ OR } (\text{NOT}(T_i) \text{ AND } S_i)$
- $\text{iguais} = \text{NOT}((T_2 \text{ XOR } S_2) \text{ OR } (T_1 \text{ XOR } S_1) \text{ OR } (T_0 \text{ XOR } S_0))$
- $\text{ACERTO (Hit)} = \text{iguais AND valido[índice]}$

O sinal ACERTO alimenta a entrada de incremento de um contador síncrono de 4 bits conectado a um display de sete segmentos hexadecimal.

2.1.5.3 Indicadores de Desempenho e Traço de 16 Acessos

A simulação do traço de 16 acessos na planilha eletrônica (parte1_planilha.xlsx) resultou nos seguintes indicadores:

- Total de Acessos: 16 | Acertos (Hits): 8 | Faltas (Misses): 8
- Taxa de Acertos (Hit Rate): $8 / 16 = 50\%$ | Taxa de Faltas (Miss Rate): $8 / 16 = 50\%$
- Faltas Compulsórias: 4 (acessos 1, 3, 9, 11) | Faltas por Conflito: 4 (acessos 6, 8, 13, 15)
- **AMAT:** $\text{AMAT} = \text{Tempo_de_Acerto} + (\text{Taxa_de_Faltas} * \text{Penalidade_de_Falta}) = 1 + (0,50 * 8) = 5$ ciclos de clock.

2.1.5.4 Tabela Completa do Traço de 16 Acessos

(parte1_planilha.xlsx)

acesso	end_hex	end_bin	rotulo	indice	offset	val_antes	rot_antes	resultado	tipo_falta	rot_apos
1	0x00	000000	000	00	0	0	-	FALTA	compulsoria	000
2	0x01	000001	000	00	1	1	000	ACERTO	-	000
3	0x02	000010	000	01	0	0	-	FALTA	compulsoria	000
4	0x03	000011	000	01	1	1	000	ACERTO	-	000
5	0x00	000000	000	00	0	1	000	ACERTO	-	000

6	0x20	100000	100	00	0	1	000	FALTA	conflito	100
7	0x21	100001	100	00	1	1	100	ACERTO	-	100
8	0x00	000000	000	00	0	1	100	FALTA	conflito	000
9	0x04	000100	000	10	0	0	-	FALTA	compulsoria	000
10	0x05	000101	000	10	1	1	000	ACERTO	-	000
11	0x06	000110	000	11	0	0	-	FALTA	compulsoria	000
12	0x07	000111	000	11	1	1	000	ACERTO	-	000
13	0x22	100010	100	01	0	1	000	FALTA	conflito	100
14	0x23	100011	100	01	1	1	100	ACERTO	-	100
15	0x02	000010	000	01	0	1	100			000
16	0x03	000011	000	01	1	1	000	ACERTO	-	000

2.1.6 Análise do Teste de Conflito com o Detector de Primos (Seção 5.6)

2.1.6.1 Regra de Geração e Comportamento na Cache de 4 Linhas

O teste aciona o detector de números primos para os valores $v = 0$ a 7, deslocando-os 3 posições à esquerda ($v \ll 3$) para formar os endereços de leitura:

v	Endereço Hex	Endereço Binário	Rótulo (A5..A3)	Índice (A2 A1)	Offset (A0)	Resultado
2	0x10	010000	010	00	0	FALTA
3	0x18	011000	011	00	0	FALTA
5	0x28	101000	101	00	0	FALTA
7	0x38	111000	111	00	0	FALTA

Explicação da Regra: Ao realizar um deslocamento de 3 bits à esquerda ($v \ll 3$), os 3 bits menos significativos dos endereços gerados (A2 A1 A0) tornam-se obrigatoriamente 000 para qualquer valor de v . Como o campo de Índice da cache de 4 linhas é composto exatamente pelos bits A2 A1, o valor do índice resulta estritamente em 00 para todos os endereços gerados.

Consequentemente, todos os quatro endereços disputam simultaneamente a linha 0 da cache. Cada novo acesso encontra um Rótulo diferente mantido na linha 0, provocando desalojamento mútuo e resultando em 4 Falhas por Conflito seguidas, mesmo que as linhas 1, 2 e 3 permaneçam totalmente ociosas. Esse comportamento evidencia a limitação do mapeamento direto (aliasing de endereços).

2.1.6.2. Análise da Expansão para uma Cache de 8 Linhas

Se a cache fosse expandida para 8 linhas mantendo o tamanho do bloco em 2 bytes:

- **Nova Divisão do Endereço:** Deslocamento (A0): $\log_2(2) = 1$ bit | Índice (A3 A2 A1): $\log_2(8) = 3$ bits | Rótulo (A5 A4): $6 - 3 - 1 = 2$ bits.
- 0x10 (010000_2): Índice = 000_2 (Linha 0), Rótulo = 01_2.
- 0x18 (011000_2): Índice = 100_2 (Linha 4), Rótulo = 01_2.
- 0x28 (101000_2): Índice = 000_2 (Linha 0), Rótulo = 10_2.
- 0x38 (111000_2): Índice = 100_2 (Linha 4), Rótulo = 11_2.

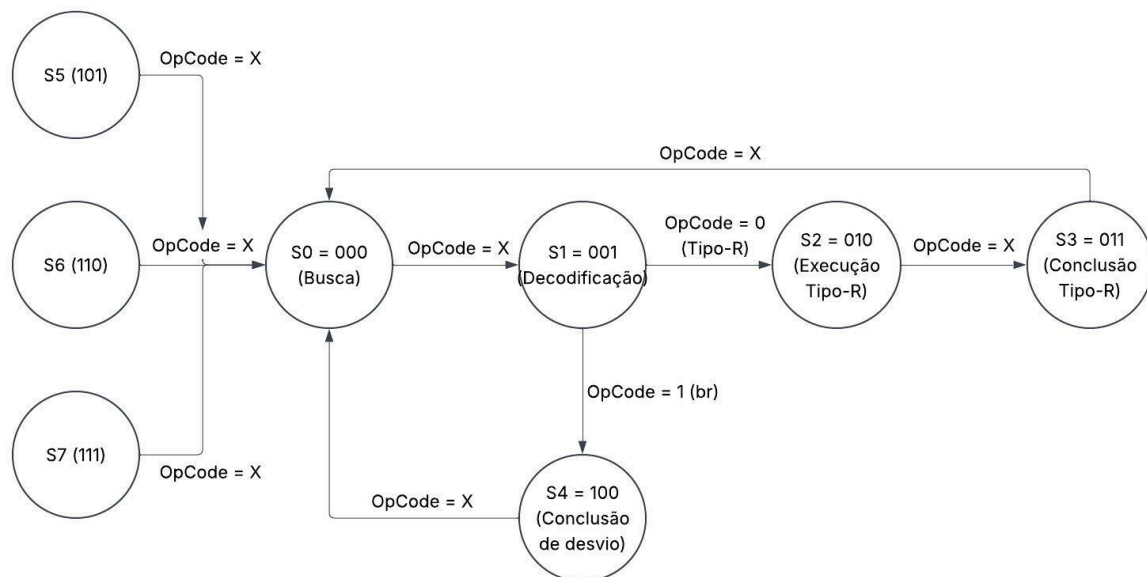
Conclusão da Expansão: Com 8 linhas, o bit A3 passa a integrar o campo de Índice. Como os valores v=2,3 possuem A3=0 e v=5,7 possuem A3=1, os acessos são distribuídos entre a Linha 0 (0x10 e 0x28) e a Linha 4 (0x18 e 0x38). O aliasing é reduzido pela metade, eliminando o conflito direto entre linhas com A3 distintos.

2.1.7 Tabela de Evidências dos Testes Práticos (ID T-01 a T-05)

ID	O que se verifica	Entradas Aplicadas	Saída Esperada / Observada	Status
T-01	Seleção de Região	A5 A4 = 00, 01, 10, 11	Apenas o respectivo pino CS atinge 1 em cada caso.	Aprovado
T-02	Erro de Paridade na RAM	Gravar 0x55 na RAM, alterar 1 bit para 0x54 no simulador e ler	O pino/LED ERRO_PARIDADE acende (1).	Aprovado
T-03	Falta Compulsória	Cache zerada (RESET = 1), ler ENDERECO = 0x00	ACERTO = 0 (linha 0 inválida).	Aprovado
T-04	Acerto no Mesmo Bloco	Após T-03 e 1 pulso de clock, ler ENDERECO = 0x01	ACERTO = 1 e contador incrementa no display.	Aprovado
T-05	Falta por Conflito	Após T-04, ler ENDERECO = 0x20	ACERTO = 0 e Rótulo da linha 0 passa a 100.	Aprovado

2.2.1 Diagrama de estados

Primeiramente, definimos como transição incondicional dos estados S5, S6 e S7 (101, 110 e 111) o estado S0 (000). Além disso, o OpCode é definido por 1 bit: 0 sinaliza instrução Tipo-R e 1 sinaliza Branch.



2.2.2 Tabela de transição

A tabela de transições referente ao diagrama de estados da seção 2.2.1 é apresentada a seguir, o estado 001 repete-se devido a bifurcação em função do OpCode, também foi adicionada a previsão dos estados excedentes:

Estado atual	Entradas	Próximo estado	Saídas
000	OpCode = X	001	$IR \leftarrow \text{Mem}[PC]; PC \leftarrow PC + 4$
001	OpCode = 0	010	$A \leftarrow \text{Reg}[rs]; B \leftarrow \text{Reg}[rt];$ $ULAOut \leftarrow PC + (\text{ext}(\text{imm}) \ll 2)$
001	OpCode = 1	100	$A \leftarrow \text{Reg}[rs]; B \leftarrow \text{Reg}[rt];$ $ULAOut \leftarrow PC + (\text{ext}(\text{imm}) \ll 2)$
010	OpCode = X	011	$ULAOut \leftarrow A \text{ op } B$
011	OpCode = X	000	$\text{Reg}[rd] \leftarrow ULAOut$
100	OpCode = X	000	ULA calcula $A - B$; se Zero, $PC \leftarrow ULAOut$
101	OpCode = X	000	-
110	OpCode = X	000	-
111	OpCode = X	000	-

2.2.3 Tabela de tempo e cálculos de ciclos

Esquematizando a tabela de tempo de cada sinal de controle em função do estado, conforme as orientações sobre o funcionamento de cada estado, temos:

2.2.3.1 S0 (Busca): $IR \leftarrow Mem[PC]$; $PC \leftarrow PC + 4$:

Portanto, $PCWrite = 1$, $MemRead = 1$, $IRWrite = 1$, $ALUSourceA = 0$ e $ALUSourceB = 01$ ($PC + 4$), e $ALUOp = 00$ (Soma).

2.2.3.2 S1 (Decodificação): $A \leftarrow Reg[rs]$; $B \leftarrow Reg[rt]$; $ULAOut \leftarrow PC + (ext(imm) \ll 2)$:

Portanto, não há escrita em PC, logo $PCSource = X$, pois é irrelevante determinar a fonte de escrita se a escrita não será realizada.

O carregamento dos registradores para A e B ocorrem combinacionalmente, $ALUSourceA = 0$ e $ALUSourceB = 10$ ($PC +$ Extensão de sinal deslocada de 2), e $ALUOp = 00$ (Soma).

2.2.3.3 S2 (Execução Tipo-R): $ULAOut \leftarrow A \text{ op } B$:

Portanto, ainda não há escrita em PC, usando a justificativa anterior para $PCSource = X$;

$ALUSourceA = 1$ e $ALUSourceB = 00$ ($A \text{ op } B$), e $ALUOp = 10$.

2.2.3.4 S3 (Conclusão Tipo-R): $Reg[rd] \leftarrow ULAOut$:

Portanto, ainda não há escrita em PC, usando a justificativa anterior para $PCSource = X$;

Como há escrita no registrador de destino, $RegWrite = 1$;

A operação desejada com A e B já foi realizada, logo, $ALUSourceA = X$, $ALUSourceB = XX$ e $ALUOp = XX$.

2.2.3.5 S4 (Conclusão de desvio): ULA calcula $A - B$; se Zero, $PC \leftarrow ULAOut$:

Há escrita condicional no PC, logo, $PCWriteCond = 1$ e $PCSource = 1$ (O valor a sobrescrever vem do endereço calculado pela ALU);

ALU realiza subtração para comparar A e B e gerar o Zero, portanto, $ALUSourceA = 1$ e $ALUSourceB = 00$, $ALUOp = 01$ ($A - B$).

Estado	PCWr	PCWrC	PCSrc	MemRd	IRWr	RegWr	SrcA	SrcB	ALUOp
S0	1	0	0	1	1	0	0	01	00
S1	0	0	X	0	0	0	0	10	00
S2	0	0	X	0	0	0	1	00	10

S3	0	0	X	0	0	1	X	XX	XX
S4	0	1	1	0	0	0	1	00	01

Portanto, a partir dessa tabela calculamos o número de ciclos de clock que cada classe de instrução (Tipo-R e beq) consome:

Classe	Estados percorridos	Número de ciclos
Tipo-R (add,sub,and,or)	S0 > S1 > S2 > S3	4
beq	S0 > S1 > S4	3

Logo, calcula-se o CPI médio das frequências de cada classe de instrução, dado pelo somatório do produto da frequência de cada classe de instrução pelo total de ciclos da mesma classe.

$$CPI_{médio} = \sum f \cdot Total_{ciclos}$$

$$CPI_{médio} = 0,7 \cdot 4 + 0,3 \cdot 3$$

$$CPI_{médio} = 2,8 + 0,9$$

$$\therefore CPI_{médio} = 3,7$$

2.2.4 Mapas de Karnaugh

Primeiramente, se constitui os mapas de Karnaugh dos flip flops tipo D em função do próximo estado que devem assumir, logo, se monta uma tabela verdade de cada flip flop conforme o estado a se seguir:

Próximo estado	Q_2	Q_1	Q_0
S0 = 000	0	0	0
S1 = 001	0	0	1
S2 = 010	0	1	0
S3 = 011	0	1	1
S4 = 100	1	0	0
S5 = 101	0	0	0
S6 = 110	0	0	0
S7 = 111	0	0	0

Note que Q_0 é o bit menos significativo do estado, e Q_2 o bit mais significativo. Além de se incluir os estados S5, S6 e S7, pois apesar de serem estados excedentes, é importante registrá-los nos flip flops para a previsão de falhas e saída no display de 7 segmentos.

Além disso, todos os flip flops são zerados nos estados de excesso, pois considera-se como retorno ao estado S0.

Logo, com base na tabela, adotou-se como variáveis para cada flip flop os estados em que eles estão ativos.

Q_0 :

	$\bar{S}_2$	S_2	$\bar{S}_3$	
$\bar{S}_1$	0	X	X	S_5
S_1	X	X	X	$\bar{S}_5$
$\bar{S}_1$	0	1	X	S_3
S_1	1	X	X	$\bar{S}_3$

$X = \text{Estados impossíveis / indefinidos}$

$\therefore Q_0 = S_1 + S_3$

$\rightarrow$ Quando não é nenhum, de fato 0.

Q_1 :

	$\bar{S}_3$	S_3	$\bar{S}_2$	
$\bar{S}_2$	0	X	X	S_6
S_2	X	X	X	$\bar{S}_6$
$\bar{S}_2$	1	X	X	S_7
S_2	0	1	X	$\bar{S}_7$

$\therefore Q_1 = S_2 + S_3$

Q_2 :

	$\bar{S}_5$	S_5	$\bar{S}_6$	
$\bar{S}_4$	0	X	X	S_0
S_4	X	X	X	$\bar{S}_0$
$\bar{S}_4$	1	X	X	S_7
S_4	0	0	X	$\bar{S}_7$

$\therefore Q_2 = S_4$

Adotou-se X como os campos em que os estados são impossíveis ou indefinidos devido a regra de Máquina de Estados Finitos (FSM) em que não se pode estar em mais de um estado simultaneamente.

Além disso, note que os mapas são semelhantes quando se posiciona as variáveis também de forma idêntica, aplicando 1 quando somente um estado é ativado e 0 quando nenhum está ativado ou quando somente um estado inválido está ativado.

Para os mapas de Karnaugh dos sinais de controle, baseamo-nos na tabela de tempo da seção 2.2.3, criando mapas de 5 variáveis (os 5 estados válidos), e escolhendo como saídas os sinais PCWrite, PCWriteSource e ALUSourceA. Aplicando X quando correspondente à tabela de tempo.

PCWrite:
 $S_0 = 1$

$\bar{S}_2$	S_2	$\bar{S}_2$
$\bar{S}_1$	0	0
S_1	0	0
$\bar{S}_1$	1	0

$S_0 = 0$

$\bar{S}_2$	S_2	$\bar{S}_2$
$\bar{S}_1$	0	0
S_1	0	0
$\bar{S}_1$	0	0

$PCWrite = S_0 \bar{S}_1 \bar{S}_2 \bar{S}_3 \bar{S}_4$

PCSource:
 $S_0 = 1$

$\bar{S}_2$	S_2	$\bar{S}_2$
$\bar{S}_1$	0	0
S_1	0	0
$\bar{S}_1$	0	0

$S_0 = 0$

$\bar{S}_2$	S_2	$\bar{S}_2$
$\bar{S}_1$	X	0
S_1	0	0
$\bar{S}_1$	0	0

$PCSource = \bar{S}_0 \bar{S}_1 \bar{S}_2 \bar{S}_3 \bar{S}_4$

ALUSourceA:
 $S_0 = 1$

$\bar{S}_2$	S_2	$\bar{S}_2$
$\bar{S}_1$	0	0
S_1	0	0
$\bar{S}_1$	0	0

$S_0 = 0$

$\bar{S}_2$	S_2	$\bar{S}_2$
$\bar{S}_1$	0	0
S_1	X	0
$\bar{S}_1$	0	0

$ALUSourceA = \bar{S}_0 \bar{S}_1 \bar{S}_3 (S_2 \bar{S}_4 + \bar{S}_2 S_4)$

2.2.5 Equações booleanas

Equações de sinal de controle: Os sinais escolhidos foram PCWrite, PCWriteSource e ALUSourceA.

- $PCWrite = S_0 \cdot \bar{S}_1 \bar{S}_2 \bar{S}_3 \bar{S}_4$
- $PCWriteSource = \bar{S}_0 \bar{S}_1 \bar{S}_2 \bar{S}_3 \cdot S_4$
- $ALUSourceA = \bar{S}_0 \bar{S}_1 \bar{S}_3 \cdot (S_2 \cdot \bar{S}_4 + \bar{S}_2 \cdot S_4)$

Equações dos flip flops tipo D: Q2 Q1 Q0

- $Q_0 = S_1 + S_3$
- $Q_1 = S_2 + S_3$
- $Q_2 = S_4$

Todas as equações foram obtidas a partir dos mapas de Karnaugh mostrados em 2.2.4.

2.2.6 Tabela do microprograma

Dada a estrutura da microinstrução, a tabela de microprograma para cada etapa do multiciclo foi montada:

Rótulo	Endereço	ULAOp	Fonte	Ação	Seq	Hexadecimal
Busca	0	00	00	01	00	0x04
Decod	1	00	10	00	10	0x22
ExecR	2	10	01	00	00	0x90
ConclR	3	00	01	10	01	0x19
ConclBEQ	4	01	01	11	01	0x5D

Para os demais espaços da memória ROM, a instrução mais segura analisada foi 0x11 (00 01 00 01), pois nada ocorre por conta de o campo Ação ser 00 e Seq = 01 aponta de volta ao início, criando o mínimo de sinais de controle de escrita.

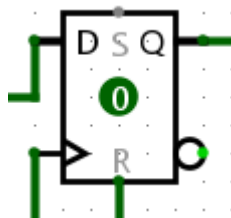
3.1 Componentes da parte I

3.2 Componentes da parte II

- Flip Flop tipo D:

Pino D utilizado para receber 8 bits a gravar do estado no flip flop, CLOCK utilizado para gravar os bits em borda de subida quando Enable ativado, WE (Write Enabled) habilita a escrita no flip flop na borda de subida do clock, RESET zera o flip flop quando ativado, SET seta o flip flop para 0xFF (inutilizado), e Q e $\bar{Q}$ são saídas dos 8 bits gravados no flip flop.

D	CLOCK	Enable	RESET	Flip flop	Q	$\bar{Q}$
AAAAAAAA	↑	1	0	AAAAAAAA	AAAAAAAA	$\overline{AAAAAAAA}$
BBBBBBBB	↓	X	X	AAAAAAAA	AAAAAAAA	$\overline{AAAAAAAA}$
AAAAAAAA	↑	X	1	00000000	00000000	11111111
BBBBBBBB	↑	0	0	AAAAAAAA	AAAAAAAA	$\overline{AAAAAAAA}$

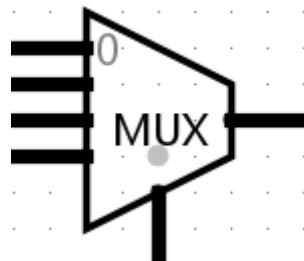


- Multiplexador de 4 entradas:

Entradas 00, 01, 10 e 11 são barramentos de mesmo número de bits a serem selecionadas pelos bits de seleção como saída, seletor de 2 bits seleciona a entrada

correspondente como saída do MUX, a saída tem o mesmo número de bits das entradas, e é escolhida pelo seletor.

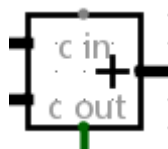
Entradas (8 bits)	Seletor	Saída (8 bits)
A, B, C e D	00	A
A, B, C e D	01	B
A, B, C e D	10	C
A, B, C e D	11	D



- Somador de 8 bits:

Entradas A e B de 8 bits a serem somadas, Carry in inicia como vai-um no bit menos significativo (inutilizado), Carry out é saída de overflow da soma, a saída Soma é a soma de 8 bits bit a bit das entradas A e B.

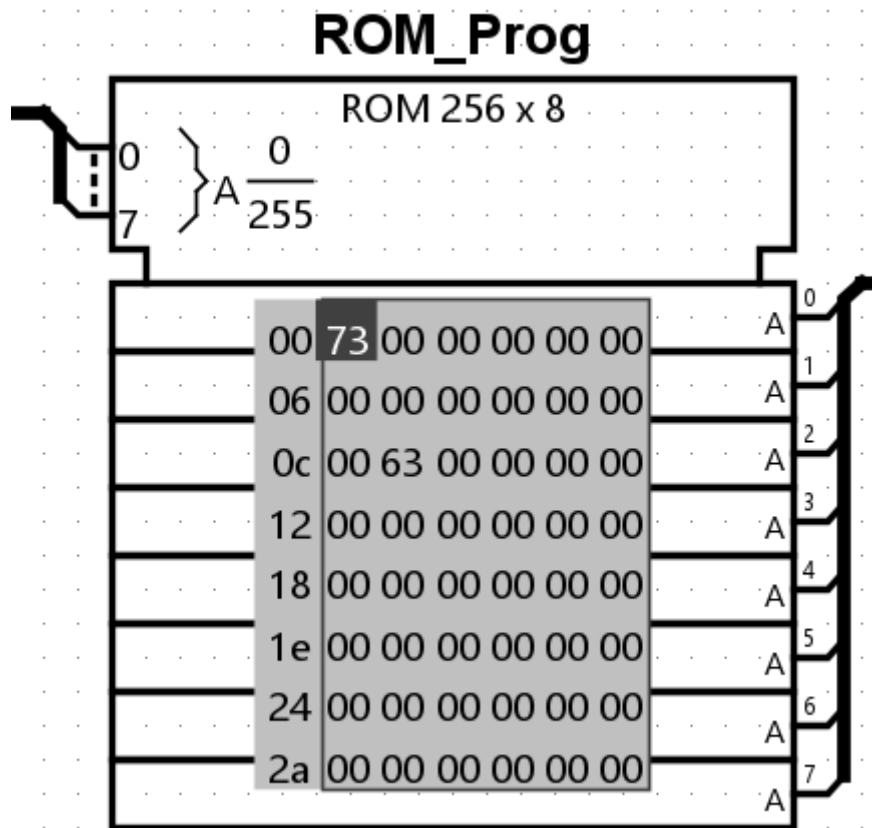
A	B	Carry out	Soma
AAAAAAAA	BBBBBBBB	0	SSSSSSSS
1AAAAAAAA	1BBBBBBB	1	SSSSSSSS



- Memória ROM de 8 bits:

Entrada de 8 bits seleciona a região da memória a ser lida, e a saída de 8 bits carregam o dado lido.

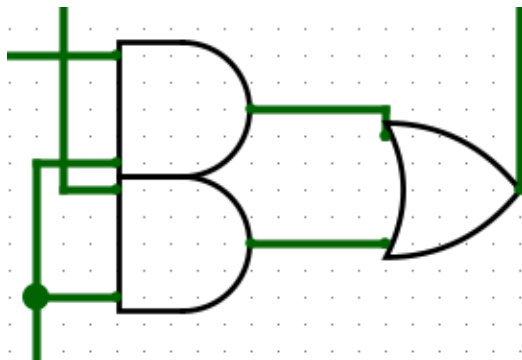
Entrada (8 bits)	Região da memória (16 bits)	Saída (8 bits)
0000 0000	0000 0000 - 0111 0011	0111 0011



- Detector da sequência “101”:

Entradas são saídas dos flip flops tipo D da versão cabeada ($Q_2 Q_1 Q_0$), com saída ligada no reset de cada um caso verifique-se sequência inválida

$Q_2 Q_1 Q_0$	RESET
101	1
110	1
111	1
Qualquer outra sequência de 3 bits	0

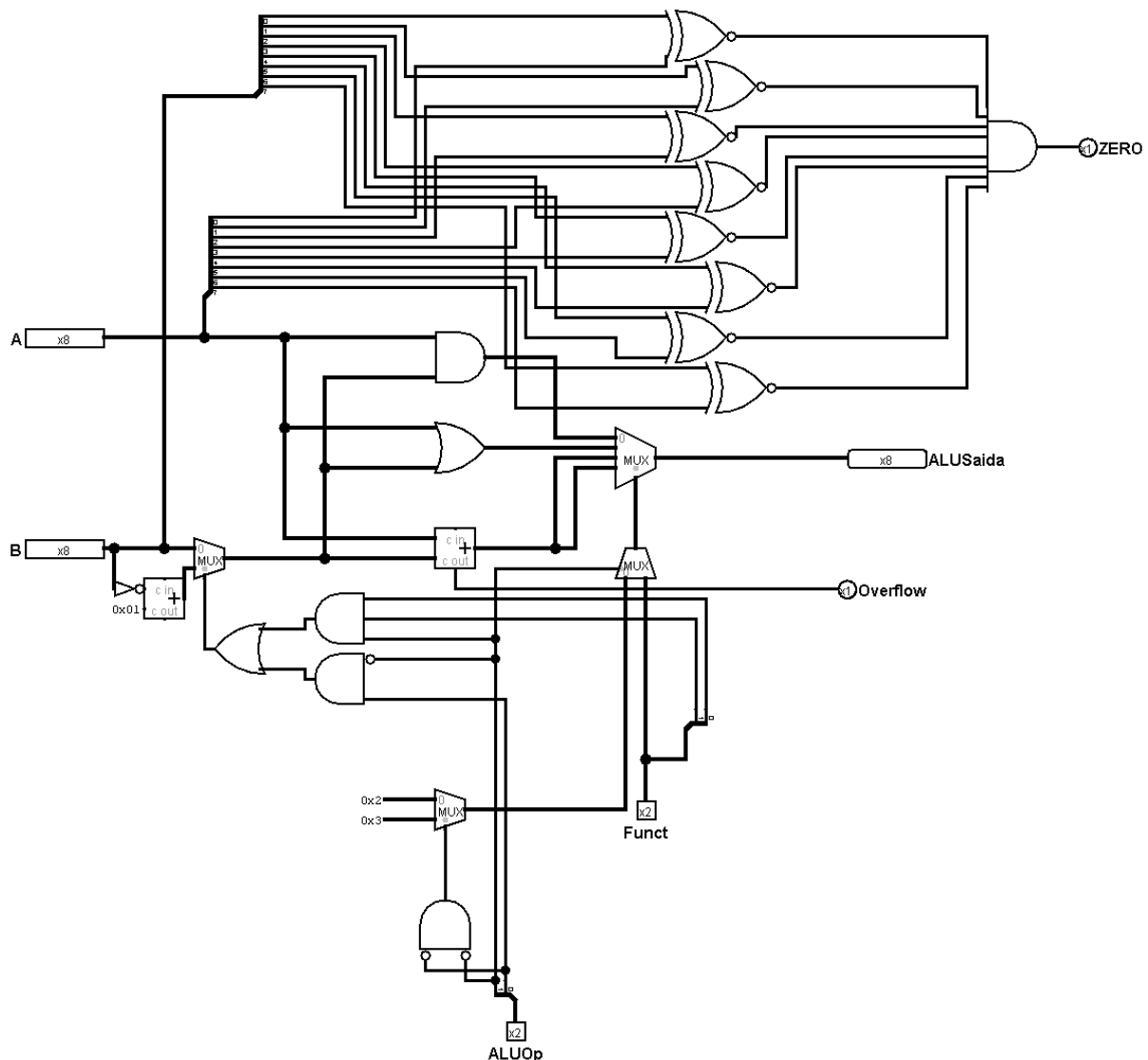


$$\text{RESET} = Q_2 \text{ AND } (Q_0 \text{ OR } Q_1)$$

- ULA de 8 bits:

Entradas são os fios A e B de 8 bits, e ULAOp e Funct de 2 bits controladores da operação com A e B, as saídas são a flag ZERO (A = B) e Overflow (overflow na soma) de 1 bit, e ALUSaída de 8 bits, resultado da operação de A e B.

ULAOp	Funct	Zero	Overflow	ALUSaída
00	XX	A = B	Excesso(A+B)	A + B
01	XX	A = B	Excesso(A+B)	$A + (\overline{B} + 1)$
10	00	A = B	Excesso(A+B)	A AND B
10	01	A = B	Excesso(A+B)	A OR B
10	10	A = B	Excesso(A+B)	A + B
10	11	A = B	Excesso(A+B)	$A + (\overline{B} + 1)$



- Extensor de sinal de 4 para 8 bits:

Entrada de 4 bits a ser convertida a 8 bits, saída é a conversão de 8 bits da entrada.

Entrada	Saída
---------	-------

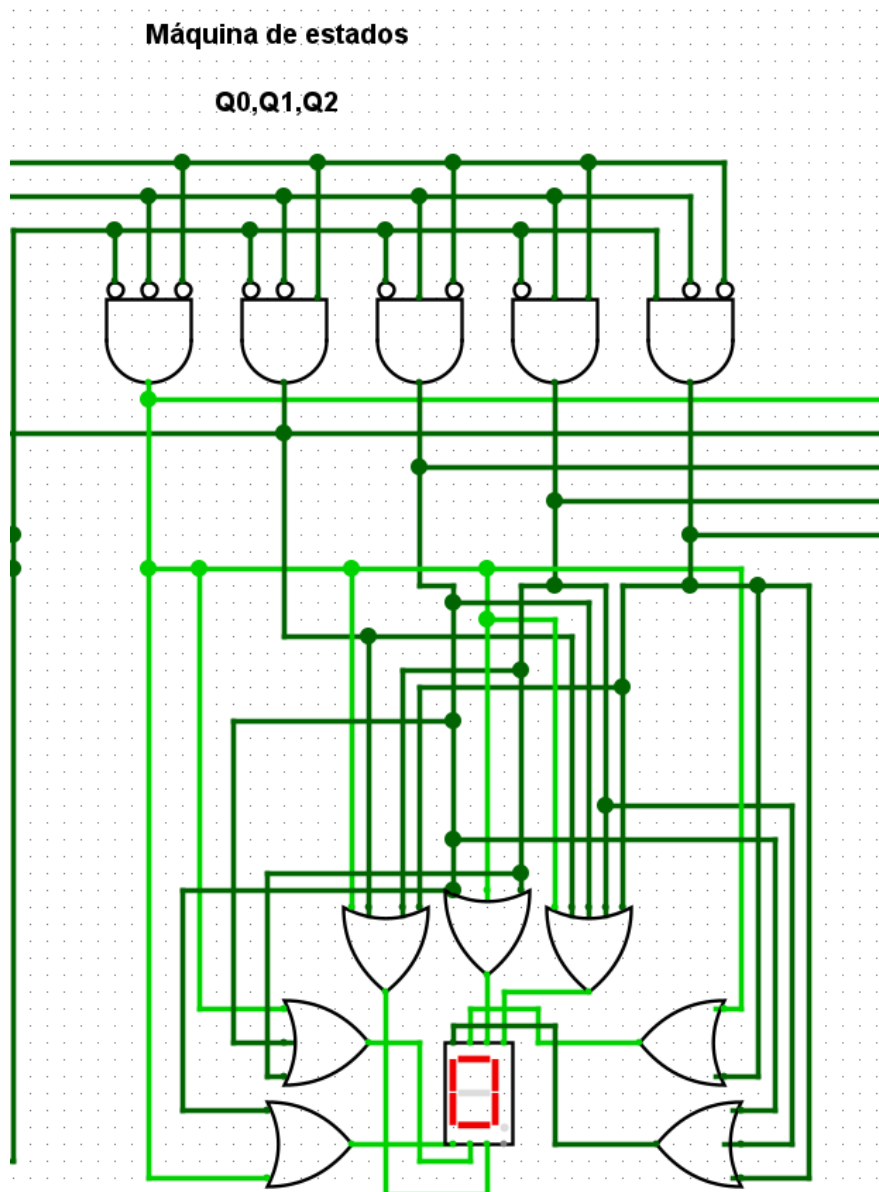
1011	1111 1011
0110	0000 0110



- Máquina de estados:

Entradas são Q_2 Q_1 Q_0 dos flip flops da UC, e as saídas são os estados formados combinacionalmente pelas entradas, além do display de 7 segmentos

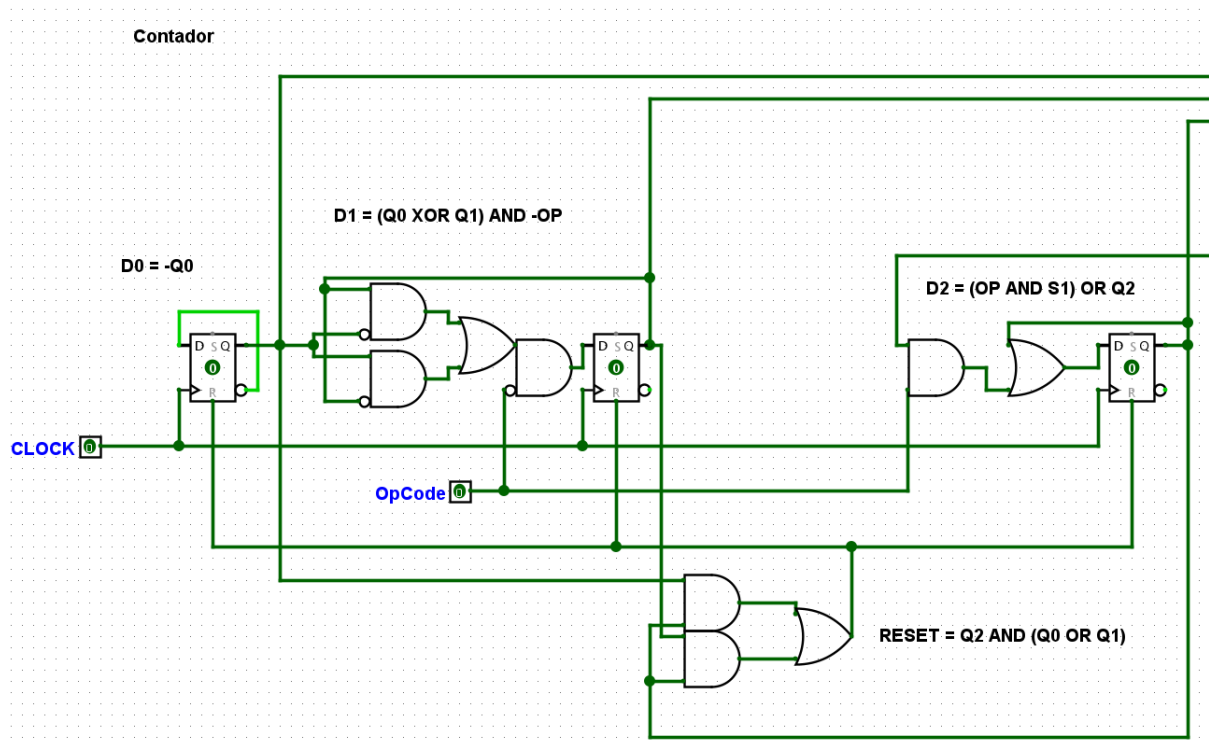
Q_2 Q_1 Q_0	Estados	Display 7 segmentos
000	S0	0
001	S1	1
010	S2	2
011	S3	3
100	S4	4



- Contador síncrono:

As entradas são o CLOCK e o OpCode do Datapath, as saídas são $Q_2 Q_1 Q_0$, produzidas pelos flips flops tipo D do contador.

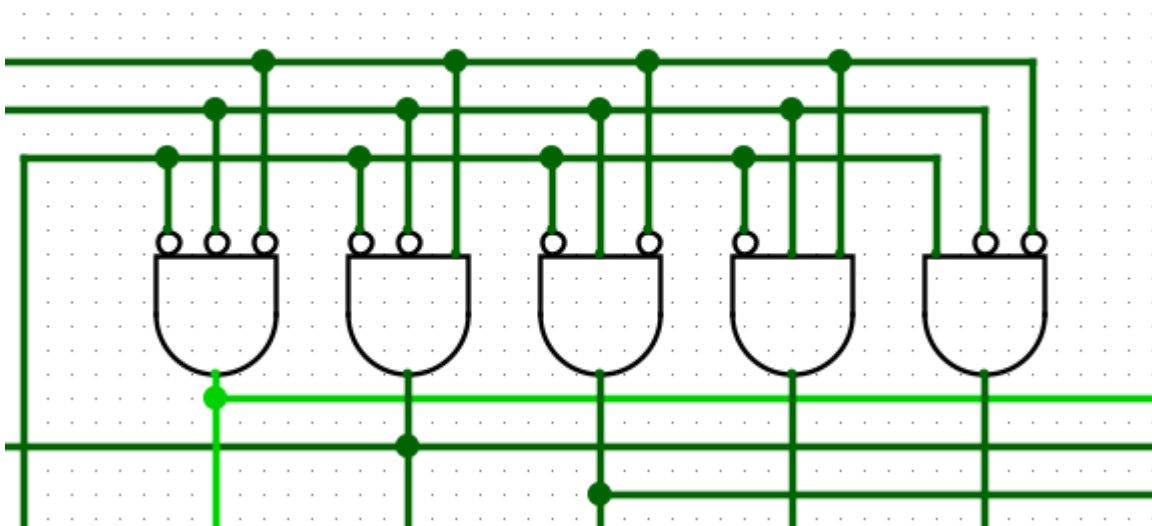
Subida de CLOCK	OpCode	$Q_2 Q_1 Q_0$
1º	X	000
2º	0	001
3º	X	010
4º	X	011
3º	1	100



- Otimização por mapas de Karnaugh:

As entradas são todos os estados que o contador de 3 bits poderia assumir (000 até 111) e as saídas são os valores dos flip flops em função dos próximos estados.

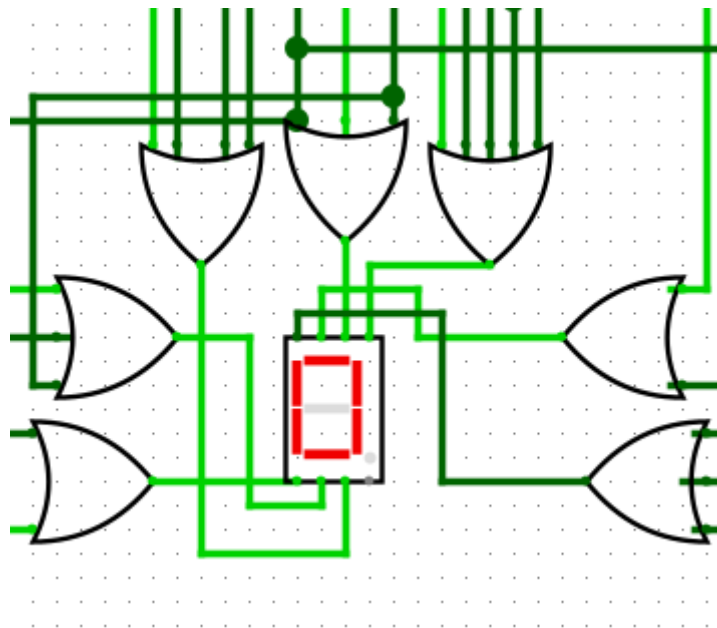
Estado	$Q_2 Q_1 Q_0$
S0	000
S1	001
S2	010
S3	011
S4	100
Qualquer outro	000



- Decodificador de 7 segmentos:

As entradas são os segmentos do visor do display, simplificadas em formas de estados da FSM, e a saída é o traço correspondente em nível igual a sua entrada.

Estado	Seg A	Seg B	Seg C	Seg D	Seg E	Seg F	Seg G
000	1	1	1	1	1	1	0
001	0	1	1	0	0	0	0
010	1	1	0	1	1	0	1
011	1	1	1	1	0	0	1
100	0	1	1	0	0	1	1



4.1 Indicadores da parte I

4.2 Comparação da parte II

Cada versão do circuito da unidade de controle possuiu suas vantagens e desvantagens na fase de projeção. Como exemplo, a velocidade de projeção: A versão cabeada demorou de 1 a 3 horas a mais para ser projetada do que a versão microprogramada, pois, se tratando de hardware puro, ficou mais difícil gerenciar o imenso cabeamento gerado pela lógica de flip flops e máquina de estados e refatorar o circuito para torná-lo funcional, ao passo que a microprogramação reduziu drasticamente o número de portas e extensão do barramento.

Ao total, o número de portas lógicas da versão cabeada foi 41, sendo 15 delas portas AND, 13 portas OR, e 13 portas inversoras NOT (excluídos da contagem os inversores dos próprios flip flops tipo D para gerar saída $\overline{Q}$). Enquanto a versão

microprogramada produziu 10 portas lógicas e 3 multiplexadores, com 5 portas AND e 5 portas NOT.

Além disso, a memória ROM da versão programada consumiu, de forma útil, 40 bits para armazenar as microinstruções válidas (AND, OR, ADD, SUB e BEQ), mas também se consumiu os 24 bits livres devido ao fator segurança do microprograma, preenchendo os 3 bytes com o código 00 01 00 01 (0x11) para que não haja nenhuma sobrescrita nos registradores do datapath, e para que o micro-contador de programa retorne à busca.

Em caso de adição de uma quinta operação na ULA, devido a estrutura do próprio datapath e da ULA, a única diferença seria a extensão do campo Funct para 3 bits, e conseqüentemente, a estrutura da ULA para escolhê-la quando Funct valer 100 e padronizar uma operação para as demais combinações ou rejeitá-las. Contudo, para ambas as versões da unidade de controle, nenhuma equação teria de ser refeita, tampouco microinstrução a ser adicionada, pois a quinta operação da ULA seria acessada por meio de Funct, e não de sinal de controle emitido pela unidade.

Dessarte, a versão cabeada é preferível em projetos de prazo de entrega longo, com o objetivo de projeção em hardware, com complexidade e nível de abstração baixo e desempenho maximizado. Em contrapartida, a versão microprogramada é preferível em projetos pequenos, e de prazo curto, com conjunto de instruções mais complexo, com prototipação e fase de testes simplificada, além de espaço compacto para projeção.

5.1 Conclusão

O projeto integrador desenvolveu as habilidades e conhecimentos teóricos aprendidos em sala de aula pelo professor, de forma a desenvolver o pensamento crítico dos estudantes a respeito de modelos e arquiteturas viáveis para cada projeto, bem como variáveis de modelagem, custo, teste e implementação.

O submódulo de memória apresentou boas evidências de funcionamento.

E a unidade de controle, tanto em forma cabeada quanto microprogramada, mostrou-se completamente funcional, emitindo os devidos sinais de controle em cada estado do datapath, além de implementar sistemas próprios de segurança para estados inválidos combacionalmente (na versão cabeada) e de forma microprogramada.

Portanto, o projeto serviu como grande atividade desenvolvidora da disciplina, contribuindo para a criação futura do projeto final, tanto no âmbito teórico como prático.

5.2 Referências

IDOETA, Ivan Valeije; CAPUANO, Francisco Gabriel. Elementos de eletrônica digital. 40. ed. São Paulo: Érica, 2007.

MEUTH, Ryan. **CSE 230 - LogiSim ALU Tutorial**. YouTube, 2013. 1 vídeo (26 min 11 s). Disponível em: <https://youtu.be/dYZ-Hwbcnq4>. Acesso em: 1 out. 2026.

APORTACION ELECTRONICA. **Contador síncrono de 3 bits - Flip Flop J-K**. YouTube, 2017. 1 vídeo (6 min 31 s). Disponível em: <https://youtu.be/FANMaccckNM>. Acesso em: 1 out. 2026.

ENGINEERING FUNDA. **Hardwired Control Unit vs Microprogrammed Control Unit: Key Differences and Comparisons | COA**. YouTube, 2022. 1 vídeo (13 min 25 s). Disponível em: <https://youtu.be/l1vBkZWYELk>. Acesso em: 1 out. 2026.

ENGINEERING FUNDA. **Microprogrammed Control Unit: Structure, Working, and Role in Computer Organization & Architecture**. YouTube, 2022. 1 vídeo (9 min 11 s). Disponível em: <https://youtu.be/5Sk5OpoT66E>. Acesso em: 1 out. 2026.

PARDO, Abelardo. **Hardwired vs microprogrammed control**. YouTube, 2013. 1 vídeo (7 min 35 s). Disponível em: <https://youtu.be/pl7mDvUwiWE>. Acesso em: 1 out. 2026.

AJAX. **Building My 16-BIT Computer. (Part 10). The Instruction Register. Logisim Evolution**. YouTube, 2024. 1 vídeo (13 min 59 s). Disponível em: <https://youtu.be/5nWZDW-n3iq>. Acesso em: 1 out. 2026.

SINGH, Mohini. **Logisim: Reading from and writing into registers in a register file**. YouTube, 2022. 1 vídeo (6 min 7 s). Disponível em: <https://youtu.be/5rnlYger55U>. Acesso em: 1 out. 2026.

BRYCE, Professor. **ALU Components in Logisim (Comp. Org. 13)**. YouTube, 2022. 1 vídeo (21 min). Disponível em: <https://youtu.be/iNUTVanB8NU>. Acesso em: 1 out. 2026.

INTERMATION. **Ep 061: D Flip-Flop Binary Counter/Timer Circuit**. YouTube, 2020. 1 vídeo (13 min 47 s). Disponível em: https://youtu.be/ts4g_NUuHAc. Acesso em: 1 out. 2026.